

PARABANK AUTOMATION TEST PLAN

Selenium WebDriver | Java | TestNG | Maven

Document Field	Details
Project Name	ParaBankAutomationFramework
Application Under Test	ParaBank Online Banking Demo Application
Prepared By	Suraj Shaw
Document Type	Automation Test Plan
Version	1.0
Date	12 June 2026
Status	Ready for Review

Wipro Capstone Project

1. Introduction

Purpose. This test plan defines the scope, objectives, approach, environment, resources, schedule, entry and exit criteria, risks, deliverables, and reporting process for automated testing of the ParaBank online banking application.

Project overview. The ParaBank Automation Framework is a hybrid Page Object Model and data-driven automation solution developed using Java, Selenium WebDriver, TestNG, Maven, Excel, Extent Reports, Jenkins, Docker, GitHub, and JIRA. The framework automates important end-to-end banking workflows and supports positive, negative, smoke, and regression testing.

2. Test Objectives

- Verify that a new customer can register successfully.
- Verify that registered users can log in using valid credentials.
- Verify validation behavior for invalid and empty login credentials.
- Verify that a logged-in user can open a new bank account.
- Verify that funds can be transferred between two different accounts.
- Verify that the account balance is updated correctly after transfer.
- Verify bill payment using multiple test-data combinations.
- Provide repeatable execution through Maven, Jenkins, and Docker.
- Generate screenshots and reports for failures and execution evidence.

3. Scope

3.1 In Scope

- User registration and generated credential storage.
- Valid login, invalid login, and empty-login validation.
- Opening a new savings account.
- Viewing account overview and retrieving account balance.
- Transferring funds between different accounts.
- Validating the remaining source-account balance.
- Bill payment using multiple datasets.
- Test execution reports, screenshots, Jenkins pipeline, and Docker build support.

3.2 Out of Scope

- Performance, load, stress, and security penetration testing.
- Database-level validation because direct database access is unavailable.
- Mobile application testing.
- Real financial transactions or production banking systems.
- Features not included in the selected capstone requirements.

4. Test Items and Planned Coverage

Test ID	Module	Scenario	Type	Priority
TC-01	Registration	Register a new customer using valid details	Positive	High
TC-02	Login	Login using valid Excel credentials	Positive	High

Test ID	Module	Scenario	Type	Priority
TC-03	Open Account	Open a new savings account	Positive	High
TC-04	Transfer Funds	Transfer funds between different accounts	Positive	High
TC-05	Bill Pay	Pay a bill using dataset 1	Data-driven	Medium
TC-06	Bill Pay	Pay a bill using dataset 2	Data-driven	Medium
TC-07	Login	Login using invalid credentials	Negative	High
TC-08	Login	Submit empty username and password	Negative	High

5. Test Approach

5.1 Automation Strategy

- Use Selenium WebDriver to automate browser interactions.
- Use TestNG annotations, assertions, priorities, listeners, and DataProvider.
- Use Page Object Model to separate page locators/actions from test logic.
- Use Excel and Apache POI for external test-data storage and retrieval.
- Use explicit waits for dynamic elements and page synchronization.
- Capture screenshots automatically when a test fails.
- Generate TestNG, Surefire, and Extent execution reports.
- Run the suite locally with Maven and through Jenkins CI.
- Use Docker to provide a repeatable build environment.

5.2 Test Types

Test Type	Purpose
Smoke Testing	Confirm that critical workflows are available and executable.
Functional Testing	Validate application behavior against expected business flow.
Regression Testing	Re-run automated scenarios after code or environment changes.
Negative Testing	Validate application handling of invalid and missing inputs.
Data-Driven Testing	Run the same bill-payment flow with multiple datasets.
Cross-Browser Testing	Supported by the framework when additional browsers are configured.

5.3 Validation Strategy

- Use TestNG hard assertions for critical validations.
- Validate page headings, confirmation messages, generated account numbers, and account balances.
- Compare the source-account balance before and after transfer.
- Verify negative login validation messages.
- Record pass/fail status in TestNG and Extent reports.

6. Test Environment

Component	Configuration
Operating System	Windows 10/11 for local execution; Jenkins agent as configured
Browser	Google Chrome
Programming Language	Java 21
Automation Tool	Selenium WebDriver
Test Framework	TestNG 7.11.0
Build Tool	Apache Maven
Driver Management	WebDriverManager
Test Data	Excel workbook under src/test/resources/testdata
Reporting	TestNG, Maven Surefire, Extent Reports, screenshots
CI/CD	Jenkins Pipeline
Containerization	Docker
Version Control	Git and GitHub
Defect Tracking	JIRA

7. Test Data Management

- Registration data is maintained in an Excel worksheet.
- A unique username is generated during registration to avoid duplicate-user conflicts.
- Generated login credentials are written to the LoginData worksheet for reuse.
- Bill-payment data is supplied through TestNG DataProvider with multiple datasets.
- Only non-sensitive demo data is stored in the repository.
- Test data should be reset or regenerated when the application state changes.

8. Entry and Exit Criteria

8.1 Entry Criteria

- ParaBank application URL is available and accessible.
- Automation project compiles successfully using Maven.
- Required Java, Maven, browser, and Jenkins tools are configured.
- TestNG suite XML files and test data are available.
- Application supports registration and account creation.

8.2 Exit Criteria

- All planned critical test cases have been executed.

- No unresolved blocker or critical automation issue prevents execution.
- All test results are available in generated reports.
- Application defects are recorded in JIRA with evidence.
- Final test execution analysis report is prepared.

9. Pass / Fail Criteria

Result	Criteria
Pass	The actual result matches the expected result and all assertions succeed.
Fail	One or more assertions fail, an unexpected exception occurs, or the expected behavior is not observed.
Skip	The test is not executed because of dependency failure, configuration issue, or intentional exclusion.
Blocked	Execution cannot continue because the application or environment is unavailable.

10. Defect Management

Application defects identified during testing will be created in JIRA using the Bug issue type. Each defect should contain the summary, description, preconditions, steps to reproduce, expected result, actual result, priority, environment, screenshots, and relevant automation logs.

Severity / Priority	Guideline
Critical	Application or major workflow is unusable and no workaround exists.
High	Core functionality fails or incorrect financial validation is allowed.
Medium	Functionality has an issue but a workaround exists or business impact is limited.
Low	Minor UI, text, or cosmetic issue with little functional impact.

11. Roles and Responsibilities

Role	Responsibility
Automation Tester - Suraj Shaw	Design framework, prepare test data, automate scenarios, execute tests, analyze results, and log defects.
Trainer / Reviewer	Review framework, test plan, reports, and project deliverables.
Jenkins Administrator	Configure JDK, Maven, Git, plugins, credentials, and pipeline execution environment.
Project Stakeholder	Review overall quality status and approve final submission.

12. Deliverables

- Automation framework source code.
- Page classes, test classes, utilities, listeners, and DataProvider.
- Excel test-data workbook and configuration files.
- TestNG XML suite files for positive and negative execution.
- Maven pom.xml.
- Jenkinsfile and Dockerfile.
- TestNG, Surefire, and Extent reports.
- Failure screenshots and execution logs.
- JIRA bug reports.
- Test case analysis report with pass percentage.
- GitHub repository and README documentation.

13. Schedule and Milestones

Activity	Planned Output	Status
Requirement analysis	Identify modules and scenarios	Completed
Framework design	Base, pages, tests, utilities, listeners	Completed
Test-data preparation	Excel and DataProvider data	Completed
Script development	Automated positive and negative cases	Completed
Local execution	Maven/TestNG execution	Completed
CI integration	Jenkins pipeline	Planned / Configured
Containerization	Dockerfile and Docker execution	Planned / Configured
Defect reporting	Create JIRA bugs with evidence	In Progress

Activity	Planned Output	Status
Final reporting	Test plan and analysis documents	In Progress

14. Risks and Mitigation

Risk	Impact	Mitigation
ParaBank application downtime	Tests cannot execute	Re-run later and record blocked status.
Chrome and driver version changes	Browser startup failure	Use WebDriverManager and maintain Selenium dependencies.
Dynamic page loading	Intermittent failures	Use explicit waits and stable locators.
Duplicate registration data	Registration fails	Generate a unique username for every execution.
State dependency between tests	Later tests may fail	Store generated data carefully and create required preconditions per test.
Jenkins agent configuration	Pipeline failure	Verify exact JDK and Maven tool names in Jenkins.
Docker browser limitations	UI tests may not start	Run Chrome in headless mode or use Selenium Grid/browser image.

15. Suspension and Resumption Criteria

15.1 Suspension Criteria

- Application is unavailable for a significant period.
- Critical environment or browser configuration prevents all tests from running.
- Test data is corrupted or missing.
- A blocking defect prevents continuation of dependent scenarios.

15.2 Resumption Criteria

- Application and environment are restored.
- Required test data and configurations are corrected.
- Blocking issues are fixed or an approved workaround is available.
- A smoke test confirms that execution can continue.

16. Metrics and Reporting

Metric	Calculation / Description
Total Tests	Number of test executions reported by TestNG.

Metric	Calculation / Description
Passed Tests	Tests completed without assertion failure.
Failed Tests	Tests with assertion failure or unexpected exception.
Skipped Tests	Tests not executed due to dependency or configuration.
Pass Percentage	$(\text{Passed Tests} / \text{Total Tests}) \times 100$
Fail Percentage	$(\text{Failed Tests} / \text{Total Tests}) \times 100$
Defect Count	Number of valid defects recorded in JIRA.

17. Assumptions and Dependencies

- ParaBank remains available and its page structure does not change significantly.
- The execution machine has network access to the application and Maven repositories.
- JDK 21, Maven, Chrome, Git, and required Jenkins plugins are installed.
- The project repository contains the correct pom.xml, suite XML files, and test resources.
- Test accounts and generated data are used only for the demo application.

18. Final Approval

This test plan will be considered approved when the reviewer confirms that the scope, approach, environment, risks, deliverables, and completion criteria are acceptable for the ParaBank capstone project.

Approval Role	Name	Signature	Date
Prepared By	Suraj Shaw	_____	13 June 2026
Reviewed By	_____	_____	_____
Approved By	_____	_____	_____